COMPUTER NETWORKS

Course Project Report

NetMonitor

Network Traffic Monitoring and Analysis Platform

Submitted By

ANUS TOQEER

Roll No: BCSF24M012

Semester: 4th

Submitted To

Madeeha Aman

Department of Computer Science

1. Introduction

1.1 Project Overview

NetMonitor is a web-based network traffic monitoring and analysis platform developed as part of the Computer Networks course. The system captures live network packets traveling through the host machine's network interface, processes and categorizes each packet, maps destination ports to known application services, maintains a persistent session log, and presents all information through an organized and filterable web dashboard.

The project is inspired by professional network analysis tools such as Wireshark but is designed specifically to demonstrate and apply fundamental networking concepts covered in the course, including packet structure, protocol identification, port-service mapping, and client-server communication.

1.2 Purpose and Motivation

The motivation behind this project is to bridge the gap between theoretical networking knowledge and practical implementation. Rather than simply studying protocols in isolation, this project required applying that knowledge to build a functional system that:

- Captures and processes real network traffic in real time
- Correctly identifies TCP, UDP and ICMP protocols
- Maps port numbers to application-layer services
- Provides meaningful statistical analysis of captured traffic
- Maintains a persistent log of all captured packets
- Presents data through a clean, user-friendly web interface

1.3 Scope

The project covers the following scope:

- Live packet capture using the Scapy library on the host machine
- Protocol identification limited to TCP, UDP and ICMP
- Port-to-service mapping for 9 common services
- Web interface using Python Flask and HTML/CSS
- Server-side filtering by protocol, source IP and destination IP
- Session logging to a CSV file with fresh start on each run
- A sample dataset for offline demonstration purposes

2. System Design and Architecture

2.1 High-Level Architecture

The system follows a client-server architecture where the Python Flask application acts as the server and the web browser acts as the client. Scapy runs as a background process on the server side, continuously capturing packets from the network interface.

Component	Role
Network Interface	Physical entry point where raw packets are captured
Scapy Library	Intercepts packets from the network interface at the OS level
handle_packet()	Processes each captured packet, extracts fields, maps services
RAM List	Stores the latest 100 packets for fast real-time display
packet_log.csv	Persistent disk storage for all packets in the session
Flask Routes	Handles HTTP requests, applies filters, calculates statistics
Jinja2 Templates	Renders Python data inside HTML for browser display
Web Browser	User interface for viewing, filtering and analyzing packets

2.2 System Flow

The following describes the complete data flow from packet capture to browser display:

1. Network packet arrives at the host machine's network interface
2. Scapy captures the raw packet and calls handle_packet() automatically
3. handle_packet() checks if the packet has an IP layer, discards if not
4. Protocol is identified — TCP, UDP or ICMP
5. Source IP, Destination IP, ports, size and timestamp are extracted
6. Destination port is looked up in the port_services dictionary
7. Packet data is appended to the in-memory captured_packets list
8. Packet data is simultaneously written to packet_log.csv on disk
9. If list exceeds 100 packets, the oldest entry is removed
10. User refreshes browser — Flask sends updated packets to HTML
11. Jinja2 renders the table with all current packets and statistics

2.3 Dual Storage Design

One of the key design decisions was implementing a two-tier storage system to balance performance with persistence:

Storage Tier	Type	Capacity	Purpose
Tier 1 — RAM List	In Memory	Latest 100 packets	Fast real-time display in browser
Tier 2 — CSV File	Disk	Unlimited — all packets	Permanent session history

2.4 Threading Model

Scapy's sniff() function is a blocking call that runs indefinitely. If called directly in the Flask main thread, it would freeze the web server and make the browser unresponsive. To solve this, Scapy runs in a separate background daemon thread:

```
sniff_thread = threading.Thread(
    target=startSniffing,
    daemon=True
)
sniff_thread.start()
```

Setting daemon=True ensures the background thread terminates automatically when the main Flask process exits, preventing orphaned processes.

3. Features and Functionality

3.1 Live Packet Capture

The core feature of NetMonitor is real-time packet capture using Scapy. When the user clicks Start Monitoring, a background thread begins intercepting all TCP, UDP and ICMP packets passing through the network interface. The Scapy filter string restricts capture to only these three protocols at the kernel level, improving efficiency.

```
sniff(
    prn=handle_packet,
    store=False,
    filter="tcp or udp or icmp"
)
```

3.2 Protocol Identification

Each captured packet is examined using Scapy's layer detection system. The haslayer() method checks whether a specific protocol layer is present in the packet stack:

Protocol	Detection Method	Ports Extracted	Common Use
TCP	packet.haslayer(TCP)	S_port, d_port	Web browsing, file transfer
UDP	packet.haslayer(UDP)	S_port, d_port	DNS lookups, streaming
ICMP	packet.haslayer(ICMP)	0, 0 (no ports)	Ping, network diagnostics

3.3 Port to Service Mapping

A Python dictionary maps destination port numbers to their corresponding application-layer service names. This is implemented as a simple lookup operation performed on every captured packet:

Port Number	Service	Protocol Layer	Description
80	HTTP	Application	Unencrypted web traffic
443	HTTPS	Application	Encrypted web traffic
53	DNS	Application	Domain name resolution
22	SSH	Application	Secure remote shell
21	FTP	Application	File transfer protocol
25	SMTP	Application	Email transmission
110	POP3	Application	Email retrieval
3306	MySQL	Application	Database communication
0	ICMP	Network	ICMP has no port numbers

3.4 Real-Time Statistics

The CalStats() function processes the current packet list and computes five statistics that are displayed as cards at the top of the dashboard. Importantly, statistics are calculated after filtering is applied, meaning filter-specific statistics are always accurate:

- Total Packets — total count of packets currently displayed
- TCP Packets — count of packets with protocol TCP
- UDP Packets — count of packets with protocol UDP
- ICMP Packets — count of packets with protocol ICMP
- Average Packet Size — total bytes divided by packet count, rounded to 2 decimal places

3.5 Server-Side Filtering

All filtering logic is implemented in Python on the server side. When a filter is applied, Flask receives the filter values via an HTTP POST request, filters the Python list using list comprehensions, and returns only the matching packets to the HTML template. No JavaScript is involved in filtering.

The source IP and destination IP filters support partial matching using Python's in operator, allowing users to type partial addresses:

```
# Partial match — typing '192.168.1' shows all matching IPs
packets = [p for p in packets
           if selected_source_ip in p['source_ip']]
```

Filters are available in all three view modes — live monitoring, stopped snapshot and full session log view.

3.6 Session Management

The system provides three distinct monitoring states:

State	Trigger	Data Displayed	Description
Live	Start Monitoring clicked	Latest 100 from RAM	Active capture in progress
Stopped	Stop Monitoring clicked	Frozen snapshot	Capture halted, data preserved
Session	View Session Log clicked	All from CSV file	Complete history from disk

3.7 Session Logging

Every packet captured during a session is written to packet_log.csv immediately after capture. The log file is automatically deleted and recreated each time app.py starts, ensuring each session begins with clean data. The CSV format includes all relevant fields: time, source IP, destination IP, protocol, packet size, source port, destination port and service.

4. Implementation Steps

The project was built incrementally in clearly defined phases. Each phase was completed and tested before moving to the next, ensuring a stable foundation at every step.

Phase 1 — Concept and Dataset

12. Studied the project requirements and identified all deliverables
13. Researched core networking concepts: packets, IP addressing, TCP/UDP/ICMP protocols, port-service relationships
14. Created the sample dataset network_data.csv manually with 40 realistic packet records covering all three protocols and multiple services
15. Verified the dataset contained all required fields: time, source IP, destination IP, protocol, packet size, source port, destination port

Phase 2 — Flask Basics

16. Installed Flask using pip install flask
17. Learned and practiced the four core Flask concepts: routing with @app.route, rendering HTML with render_template, receiving form data with request.form.get, and GET vs POST methods
18. Built and tested a Hello World Flask application to confirm the server was working
19. Practiced passing Python lists and dictionaries to HTML using Jinja2 template syntax

Phase 3 — Python Backend

20. Built the load_packets() function to read the CSV dataset using csv.DictReader
21. Implemented the port_services dictionary mapping 9 port numbers to service names
22. Added automatic service lookup inside load_packets() using the .get() method with Unknown as default
23. Built the CalStats() function to calculate total, per-protocol counts and average size
24. Wrote a test print loop to verify all 40 packets loaded correctly with the service field added

Phase 4 — HTML Interface

25. Built the complete index.html structure: header, controls, statistics grid, filter panel, packet table and system log
26. Applied CSS dark theme with black and grey base colors and green, blue and orange accent colors for protocol badges
27. Used Google Fonts to load Orbitron, Share Tech Mono and Exo 2 typefaces for a professional appearance
28. Implemented Jinja2 loops and conditionals for the packet table and protocol badge coloring

Phase 5 — Connecting Frontend and Backend

29. Connected the Flask route to render index.html with packets and stats data
30. Implemented the filter form with hidden action field to distinguish Start, Stop and Filter button clicks
31. Added server-side filtering using Python list comprehensions for all three filter types

32. Ensured filter selections persist after applying by passing them back to the template as variables

Phase 6 — Live Packet Capture

33. Installed Scapy using pip install scapy
34. Installed Npcap on Windows with WinPcap API-compatible mode enabled
35. Replaced CSV loading with Scapy's sniff() function using a background daemon thread
36. Implemented handle_packet() to process each captured packet and extract all required fields
37. Added the filter='tcp or udp or icmp' parameter to Scapy for efficient kernel-level filtering

Phase 7 — Bug Fixing and Polish

38. Fixed the Stop Monitoring bug where packets continued changing after stopping by adding the is_monitoring check inside handle_packet() and implementing a frozen snapshot
39. Added the View Session Log feature loading complete history from CSV
40. Fixed the NoneType crash in CalStats() by adding a safety check for empty packet_size values
41. Added automatic CSV log clearing on application startup for clean sessions
42. Added auto-redirect after Start Monitoring to allow Scapy time to capture first packets before rendering

5. Challenges Faced and Solutions

Challenge 1 — Flask and Scapy Conflict (Threading Issue)

Problem	When Scapy's sniff() function was called directly inside the Flask route, the web server froze completely. The browser could not load any pages because sniff() is a blocking call that runs indefinitely.

Solution	Moved Scapy's sniff() into a separate background daemon thread using Python's threading module. The thread runs independently alongside Flask, allowing both to operate simultaneously without blocking each other.

Challenge 2 — Stop Monitoring Not Working

Problem	After clicking Stop Monitoring, packets continued appearing on every page refresh. The is_monitoring flag was set to False correctly but the Scapy thread kept running and adding to the captured_packets list.
Solution	Added a check at the very beginning of handle_packet() to immediately return without processing if is_monitoring is False. Additionally implemented a stopped_packets snapshot variable that freezes the packet list at the exact moment Stop is clicked, ensuring the displayed data never changes after stopping.

Challenge 3 — ICMP Packets Not Captured

Problem	After implementing live capture, TCP and UDP packets appeared correctly but ICMP packets were completely absent even when running ping commands.
Solution	The issue was that Scapy on Windows requires Npcap to capture ICMP packets at the raw socket level. After installing Npcap with WinPcap API-compatible mode enabled and restarting the terminal, ICMP packets were captured correctly. Additionally discovered that pinging from the same machine sometimes causes issues, so external addresses like 8.8.8.8 were used for testing.

Challenge 4 — Page Showing Empty Table After Start

Problem	When the Start Monitoring button was clicked, the page reloaded immediately showing an empty table. Packets only appeared after clicking Reset or refreshing manually because Flask rendered the response before Scapy had time to capture any packets.
Solution	Added a redirect after starting monitoring using Flask's redirect and url_for functions. This sends the browser to a fresh GET request instead of rendering immediately, giving the Scapy thread a fraction of a second to capture initial packets before the page loads.

6. Testing and Verification

6.1 Generating Test Traffic

Since the system monitors the host machine's own network traffic, specific actions were performed to generate each protocol type for testing:

Protocol	Method Used	Command / Action	Expected Result
TCP	Browser navigation	Open websites in browser	TCP packets to port 80 or 443
UDP	DNS lookup	nslookup google.com	UDP packets to port 53
ICMP	Ping command	ping 8.8.8.8 -n 10	ICMP packets with port 0

6.2 Filter Testing

Each filter type was tested independently and in combination:

- Protocol filter — selected TCP only, verified only TCP rows appeared in table and statistics updated accordingly
- Source IP filter — typed partial address 192.168.1, verified all matching IPs appeared
- Destination IP filter — typed 8.8.8.8, verified only packets to that address appeared
- Combined filters — applied Protocol UDP and Destination 1.1.1.1 simultaneously, verified intersection
- Reset button — verified all filters cleared and full packet list restored

6.3 Session Log Testing

The session log feature was verified through the following test sequence:

43. Started monitoring and generated traffic for approximately 60 seconds
44. Verified packet_log.csv was being created and growing in size
45. Clicked Stop Monitoring and verified table froze
46. Clicked View Session Log and verified all packets from CSV loaded including those beyond the 100-packet RAM limit
47. Applied filters within session log view and verified they worked correctly
48. Restarted the application and verified packet_log.csv was deleted and recreated fresh

7. Future Work and Possible Enhancements

7.1 Real-Time Auto-Update

Currently the user must manually refresh the browser to see new packets. A future enhancement would implement WebSockets using Flask-SocketIO to push new packet data to the browser instantly without any page reload. This would make the display truly real-time like Wireshark.

7.2 Visual Analytics and Charts

Adding graphical visualizations would significantly improve the analytical capability of the system. Potential charts include:

- Protocol distribution pie chart showing TCP vs UDP vs ICMP percentages
- Packet rate timeline graph showing packets per second over time
- Top source and destination IPs bar chart
- Service usage breakdown showing which services generated the most traffic

7.3 Multiple Interface Support

The current implementation captures from the default network interface. A future version could detect all available network interfaces and allow the user to select which one to monitor, making it useful on machines with both WiFi and Ethernet connections.

7.4 Alert System

An anomaly detection and alerting feature could be added to notify the user when unusual patterns are detected, such as an unusually high number of packets from a single source IP, repeated ICMP requests which could indicate a ping flood, or connections to suspicious ports.

7.5 Extended Protocol Support

The current system supports only TCP, UDP and ICMP. Future versions could add support for ARP, DNS response parsing, HTTP request parsing to extract URLs and methods, and TLS/SSL detection.

7.6 Export Functionality

Adding the ability to export captured packet data in multiple formats such as PDF reports, Excel spreadsheets or PCAP files compatible with Wireshark would make the tool more versatile for network analysis and documentation.

7.7 Authentication

Since the tool captures sensitive network data, adding user authentication with a login system before accessing the dashboard would be an important security enhancement for any production deployment.

8. Conclusion

NetMonitor successfully demonstrates the core concepts of network traffic monitoring through a fully functional web-based application. The project integrates live packet capture, protocol identification, port-service mapping, real-time statistics, persistent logging and server-side filtering into a cohesive system built entirely from scratch using Python, Flask and Scapy.

The development process provided hands-on experience with several important concepts from the Computer Networks course including the structure of network packets at the IP layer, the differences between TCP UDP and ICMP in terms of connection type, port usage and typical applications, the relationship between well-known port numbers and application layer services, and the mechanics of how network monitoring tools access raw packet data.

Beyond networking concepts, the project also required applying software engineering practices such as multi-threaded programming to run Scapy alongside Flask, client-server communication through the HTTP request-response cycle, data persistence using file I/O, and defensive programming with proper error handling.

The bonus implementation of live packet capture using Scapy elevates the project beyond a simple data display tool into a genuine network monitoring system comparable in concept to professional tools like Wireshark, demonstrating both technical capability and a thorough understanding of the underlying networking principles.

9. References

Libraries and Frameworks

Library / Tool	Version	Purpose	Source
Python	3.8+	Core programming language	python.org
Flask	3.x	Web framework and HTTP server	flask.palletsprojects.com
Scapy	2.5+	Live packet capture and analysis	scapy.net
Jinja2	3.x	HTML templating engine	jinja.palletsprojects.com
Npcap	1.7+	Windows packet capture driver	npcap.com
Google Fonts	—	Web typography	fonts.google.com
threading	stdlib	Background thread management	Python standard library

csv	stdlib	CSV file reading and writing	Python standard library
os	stdlib	File system operations	Python standard library
datetime	stdlib	Timestamp generation	Python standard library

Technical References

- RFC 791 — Internet Protocol (IP) Specification
- RFC 793 — Transmission Control Protocol (TCP) Specification
- RFC 768 — User Datagram Protocol (UDP) Specification
- RFC 792 — Internet Control Message Protocol (ICMP) Specification
- IANA Service Name and Transport Protocol Port Number Registry — iana.org
- Flask Official Documentation — flask.palletsprojects.com/en/3.0.x
- Scapy Official Documentation — scapy.readthedocs.io
- Npcap Official Documentation — npcap.com/guide

Conceptual References

- Forouzan, B.A. — Data Communications and Networking, 5th Edition, McGraw-Hill
- Tanenbaum, A.S. — Computer Networks, 5th Edition, Pearson
- Kurose, J.F. and Ross, K.W. — Computer Networking: A Top-Down Approach, 8th Edition